ITE083 — IT Project Management

FINAL PROJECT DOCUMENT

Fixko — Campus Facility Maintenance Reporting System

Industry Chosen: Education — School / University Facilities

Date Submitted: October ___, 2026

Class Code: [YOUR CLASS CODE]

Submitted by: [Group Members — see Section 3]

Table of Contents

1. Target Industry
2. Project Charter
3. Group Members and Roles
4. Software Development Life Cycle
5. Data Dictionary
6. Entity Relationship Diagram (ERD)
7. Summary, Conclusion, & Recommendations
8. Curriculum Vitae (attached at the last page)

1. Target Industry

Industry Chosen: Education — School / University Facility Maintenance

Industry Overview

Schools and universities depend heavily on their physical facilities — classrooms, laboratories, offices, restrooms, and campus equipment. For classes and daily operations to run smoothly, damaged equipment and facility problems must be discovered, reported, fixed, and verified quickly. In most schools this process is still done verbally or on paper, which causes delays, lost records, and wasted maintenance effort. Fixko digitizes this process: any member of the campus community can report a facility problem in seconds with photo proof, the administration can review and assign it, and maintenance personnel can update and resolve it — with every step visible to the reporter.

Common Problems in the Industry

Verbal and paper-based reporting — students must personally visit the maintenance or admin office to report a problem, and many issues are reported multiple times or reported to the wrong person.

No status visibility — after a report is filed, the reporter has no way of knowing whether it is being reviewed, being fixed, or completely forgotten.

Scattered and unreliable records — handwritten logs get lost or duplicated, so there is no single centralized database of facility problems and their history.

Slow and unfair assignment — the admin has no clean overview of open reports, so urgent repairs wait while less important ones get attention.

No accountability or feedback loop — repairs are never rated, so no one knows how effective and complete the maintenance work actually is.

Fake or unverifiable reports — paper reports have no photo proof, and even digital submissions can be abused with AI-generated images.

Target Users or Beneficiaries

Students (Reporters): Can submit facility reports in seconds from anywhere, attach real photo proof, and follow the live status of every report they file instead of wondering whether anyone is acting on it.

Maintenance Personnel: Receive clearly assigned reports with the location, item, and photo already attached, so no job goes unowned or forgotten — and their unaddressed work is visible to the admin.

School Administrators: Get a centralized overview of all facility problems, priority and duplicate warnings, clean assignment tools, and dashboards that show which equipment keeps breaking.

The University / Department: Builds a permanent, searchable record of all facility issues and repairs — useful for budgeting, equipment replacement planning, and quality audits.

2. Project Charter

Project Title: Fixko — Campus Facility Maintenance Reporting System

Project Purpose

Fixko exists because campus facility reporting is still verbal and paper-based, which causes lost records, repeated reports, unnoticed repairs, and no accountability. The project replaces that manual loop with one digital platform that connects the reporter, the administrator, and the maintenance personnel from the first report to the final fix.

Project Scope

In Scope (Included):

Role-based accounts: User (reporter), Maintenance personnel, and Admin with secure token authentication.

Report submission with room/equipment selection, category, description, priority, and photo upload — with automatic AI-generated-image detection and refusal.

Full status workflow: Pending !’ Under Review !’ In Progress !’ Under Maintenance !’ Resolved, with a complete status history log.

Admin dashboard: all reports, Report Summary charts, Done Works archive, and user management (warn, suspend, restore, delete).

Report assignment to maintenance personnel (only the admin assigns; resolved reports are locked).

Per-report comment thread shared by reporter, maintenance, and admin.

Role-aware notification system with unread badges, archive, and bulk actions.

Reporter feedback and 1–5 rating after resolution.

Discipline system: flagged reports auto-warn, then auto-suspend (1 week), then auto-lock the accountable account.

Out of Scope (Excluded)

Real-time chat between users outside the report comment thread.

Mobile native app (the system is web-responsive).

Payments, budgeting, or inventory purchasing.

Integration with the school's official student information system.

Automatic scheduling of maintenance work shifts.

Main Project Objective

To deliver one centralized web system that lets any campus member report facility problems with verifiable photo proof and lets the administration assign, track, and resolve every report until it is verified fixed — eliminating lost paper records and giving reporters real visibility into progress.

Key Stakeholders

Students and other campus reporters — the primary source of facility reports.

Maintenance personnel — execute the repairs and keep the statuses moving.

School administrators — oversee reports, assign work, and maintain user accounts.

Office of the department / student organizations — consumers of the summary data for planning and audits.

3. Group Members and Roles

[To be filled in by the group: table of Name, Role, and Responsibility — Project Leader listed first, other members in alphabetical order by surname.]

4. Software Development Life Cycle

[Insert SDLC Model Diagram Image Here]

Chosen SDLC Model: Iterative (Incremental) Model

Justification

Fixko was built as separate features that each depend on the previous one — authentication first, then reports, then assignment and statuses, then comments and notifications. The iterative model let the group build and show a working version of the system after each feature cycle, get immediate feedback from classmates and end users, and fix integrations between the Laravel API and the Next.js frontend before moving to the next feature. A pure Waterfall approach was risky because role-based UI decisions (especially the three different user experiences) could not be finalized until earlier features were actually used.

Key Phases

Planning: The group identified the target pain points of campus reporting, agreed on the three user roles, defined the feature set (reporting, workflow, notifications, feedback), and set the technology stack and milestone schedule per member role.

Design: The database schema (users, rooms, equipment, reports, comments, status histories, feedback, notifications) and the Laravel REST API endpoints were designed first, together with wireframes of the three role-specific experiences and the dark/light theme system.

Development: Coding proceeded in cycles: (1) authentication and accounts, (2) report submission with photo upload and ticket codes, (3) assignment and status workflow, (4) comments and notifications, (5) flags, warnings, and dashboards. Each cycle ended with a working integrated demo.

Testing: Each role was tested end-to-end after every cycle: reporter submission !’ admin assignment !’ maintenance status updates !’ feedback. Edge cases like duplicate reports, fake/AI photos, resolved-report locking, and notification routing across roles were deliberately tested and fixed.

Deployment: The system runs on local XAMPP (Apache + MySQL) with the Laravel backend serving the Next.js frontend; the final build was packaged for a live demo, with cloud deployment identified as the next step.

5. Data Dictionary

Google Sheets Link: [Insert actual link here once uploaded]

The Fixko database is implemented in MySQL. The tables below list the fields of each table with their data types, descriptions, and constraints.

Table: users

Table	Field Name	Data Type	Description	Constraints
users	id	BIGINT (Integer)	Unique user ID	Primary Key, Auto-increment
users	name	VARCHAR	Full display name	Required
users	email	VARCHAR	Login email address	Unique, Required
users	school_id	VARCHAR	School ID number for verification	Unique, Required
users	password	VARCHAR	Hashed password (bcrypt)	Required, Not Null
users	role	ENUM/STRING	Account role	user / admin / maintenance (default: user)
users	account_status	STRING	Account discipline state	active / suspended / locked (default: active)
users	status_until	TIMESTAMP	End of suspension period	Nullable
users	warnings_count	INT	Count of formal warnings received	Default: 0
users	department	VARCHAR	Department / program of the user	Nullable
users	created_at / updated_at	TIMESTAMP	Record timestamps	Auto-managed

Table: rooms

Table	Field Name	Data Type	Description	Constraints
rooms	id	BIGINT (Integer)	Unique room ID	Primary Key, Auto-increment
rooms	building	VARCHAR	Name of the campus building	Required
rooms	floor	VARCHAR	Floor location of the room	Nullable
rooms	name	VARCHAR	Room name/number	Required
rooms	code	VARCHAR	Short room code	Unique, Nullable
rooms	department	VARCHAR	Department owning the room	Nullable

Table: equipment

Table	Field Name	Data Type	Description	Constraints
equipment	id	BIGINT (Integer)	Unique equipment ID	Primary Key, Auto-increment
equipment	room_id	BIGINT	Room the equipment belongs to	FK !' rooms.id, Restrict on delete
equipment	name	VARCHAR	Name of the equipment / item	Required
equipment	category	VARCHAR	Equipment category	Nullable
equipment	asset_tag	VARCHAR	Inventory asset tag	Unique, Nullable
equipment	condition	ENUM	Current condition	new / good / fair / poor / broken (default: good)
equipment	quantity	INT	Number of units	Default: 1
equipment	description	TEXT	Free-text details	Nullable

Table: reports

Table	Field Name	Data Type	Description	Constraints
reports	id	BIGINT (Integer)	Unique report ID (ticket code FX-XXXX)	Primary Key, Auto-increment
reports	reporter_id	BIGINT	User who submitted the report	FK !' users.id, Cascade on delete
reports	reporter_school_id_snapshot / reporter_email_snapshot	VARCHAR	Reporter info preserved even if the account changes	Required
reports	room_id	BIGINT	Location of the reported issue	FK !' rooms.id, Restrict on delete
reports	equipment_id	BIGINT	Specific equipment involved, if any	FK !' equipment.id, Nullable, Null on delete
reports	category	VARCHAR	Type of issue (e.g., damage, missing)	Nullable
reports	item_name	VARCHAR	Name of the affected item	Nullable
reports	condition	VARCHAR	Condition of the item when reported	Nullable
reports	quantity	INT	Affected units	Nullable
reports	description	TEXT	Full description of the issue	Required, Not Null
reports	priority	ENUM/STRING	Urgency of the repair	low / medium / high / critical (default: medium)
reports	status	ENUM/STRING	Current workflow stage	pending / under_review / in_progress / under_maintenance / resolved (default: pending); indexed with priority
reports	handled_by	BIGINT	Maintenance personnel assigned	FK !' users.id, Nullable, Null on delete
reports	photo_url	VARCHAR	Path of the uploaded photo proof	Nullable
reports	is_flagged	BOOLEAN	Report flagged for violation	Default: false
reports	flag_reason / flag_note / flagged_at	VARCHAR / TEXT / TIMESTAMP	Details of the flag	Nullable
reports	resolved_at	TIMESTAMP	When the report was resolved	Nullable

Table: comments

Table	Field Name	Data Type	Description	Constraints
comments	id	BIGINT (Integer)	Unique comment ID	Primary Key, Auto-increment
comments	report_id	BIGINT	Report the comment belongs to	FK !' reports.id, Cascade on delete
comments	author_id	BIGINT	User who wrote the comment	FK !' users.id, Cascade on delete
comments	message	TEXT	Comment content	Required, Max 2000 chars

Table: ticket_status_histories

Table	Field Name	Data Type	Description	Constraints
ticket_status_histories	id	BIGINT (Integer)	Unique history entry ID	Primary Key, Auto-increment
ticket_status_histories	report_id	BIGINT	Report whose status was changed	FK !' reports.id, Cascade on delete
ticket_status_histories	old_status / new_status	VARCHAR	Status before and after the change	new_status required; old_status nullable (first entry)
ticket_status_histories	changed_by	BIGINT	User who made the change	FK !' users.id, Restrict on delete
ticket_status_histories	changed_at	TIMESTAMP	When the change happened	Auto (defaults to current)

Table: feedback

Table	Field Name	Data Type	Description	Constraints
feedback	id	BIGINT (Integer)	Unique feedback ID	Primary Key, Auto-increment
feedback	report_id	BIGINT	Resolved report being rated	FK !' reports.id, Unique, Cascade on delete
feedback	user_id	BIGINT	Reporter giving the feedback	FK !' users.id, Cascade on delete
feedback	rating	TINYINT	Repair rating	1–5, Required
feedback	comment	TEXT	Optional written feedback	Nullable, Max 1000 chars

Table: repair_history

Table	Field Name	Data Type	Description	Constraints
repair_history	id	BIGINT (Integer)	Unique history record ID	Primary Key, Auto-increment
repair_history	equipment_id	BIGINT	Equipment that was repaired	FK !' equipment.id, Cascade on delete
repair_history	report_id	BIGINT	Report that triggered the repair	FK !' reports.id, Nullable, Null on delete
repair_history	user_id	BIGINT	Personnel who performed the action	FK !' users.id, Restrict on delete
repair_history	action	VARCHAR	Action performed (e.g., inspection, replacement)	Required
repair_history	notes	TEXT	Details of the repair	Nullable
repair_history	cost	DECIMAL(10,2)	Cost of the repair	Nullable

Table	Field Name	Data Type	Description	Constraints
repair_history	repaired_at	DATE	Date of the repair	Nullable

Table: notifications

Table	Field Name	Data Type	Description	Constraints
notifications	id	BIGINT (Integer)	Unique notification ID	Primary Key, Auto-increment
notifications	user_id	BIGINT	Recipient of the notification	FK !’ users.id, Cascade on delete
notifications	message	TEXT	Notification content and role-aware details	Required
notifications	is_read	BOOLEAN	Whether the recipient has read it	Default: false
notifications	archived_at	TIMESTAMP	When the notification was archived	Nullable, Indexed
notifications	related_report_id	BIGINT	Report the notification is about, for deep-linking	FK !’ reports.id, Nullable, Null on delete

6. Entity Relationship Diagram (ERD)

[Insert ERD Image Here]

Description of Relationships

A user can submit many reports; each report is submitted by one reporter (reports.reporter_id !’ users.id).

A report is located in one room; a room can have many reported issues (reports.room_id !’ rooms.id).

A room can contain many equipment items; a report may reference one specific equipment (equipment.room_id !’ rooms.id; reports.equipment_id !’ equipment.id).

A report may be assigned to one maintenance personnel; one personnel can handle many reports (reports.handled_by !’ users.id).

A report can have many comments; each comment has one author (comments.report_id !’ reports.id; comments.author_id !’ users.id).

A report has many status-history entries recording every transition, and one resolved report can have at most one feedback entry (rating 1–5).

A user can receive many notifications; a notification may link to one report (notifications.user_id !’ users.id; notifications.related_report_id !’ reports.id).

Equipment can have many repair-history records, optionally tied to the report that caused the repair.

7. Summary, Conclusion, & Recommendations

Summary

Fixko is a full-stack web system that digitizes campus facility maintenance reporting. It replaces verbal and paper-based reporting with one centralized platform where students (reporters), maintenance personnel, and school administrators each play a defined role. A reporter submits a report with room, item, description, priority, and verifiable photo proof — with AI-generated images automatically detected and refused — and receives a unique ticket code (FX-XXXX) whose live status they can track at any time. The administrator reviews all incoming reports, sets priority and duplicate warnings, assigns each report to a specific maintenance personnel, and can flag abusive reports, which automatically disciplines the accountable account with escalating warnings, suspension, and lock. Maintenance personnel move their assigned reports through the defined workflow (Pending !' Under Review !' In Progress !' Under Maintenance !' Resolved) while communicating with the reporter inside a per-report comment thread. Role-aware notifications keep every user informed of assignments, status changes, and replies, and after resolution the reporter rates the completed repair from 1 to 5. Each role is served by a tailored dashboard that summarizes open work, critical items, and completion.

Conclusion

Fixko achieves its main objective: it converts a manual, error-prone reporting process into one centralized, verifiable, and fully tracked digital workflow. Lost paper records are eliminated because every report lives in the database with a permanent status history; redundant reports are reduced because reporters can see their ticket's live progress; and accountability is enforced because every report has a named assignee and every misuse has an escalating consequence. Building the system also taught the group the realities of full-stack integration — connecting Laravel Sanctum authentication with a Next.js frontend, designing role-based authorization on both the API and the UI layers, and fixing flow-level bugs that only appear across different user points of view. The biggest lessons were the value of version control, early end-to-end role testing, and designing the API contract before coding the interface.

Recommendations

Add an analytics expansion to the dashboards: resolve-time trends, equipment that breaks repeatedly, and department-level summaries to guide budgeting and equipment replacement.

Develop a mobile-friendly PWA or native app with push notifications so facility problems can be reported and updated instantly from phones.

Deploy the system to a cloud hosting platform with scheduled database backups so it serves the whole campus rather than only the school intranet.

Add email notifications and optional two-factor authentication for admin accounts to further strengthen security.

Integrate a campus map so reporters can pick the exact equipment location, reducing wrong-room reports.

For future developers: keep CI/CD with automated end-to-end role tests before merging, and document API contract changes in the repository to avoid breaking the separate frontend.